

AIM: Component and Arduino Introduction

Component and Arduino Introduction

The Arduino Uno is a versatile and widely used microcontroller board that acts as the heart of many electronics projects. It is an open-source platform that allows users to design and develop interactive electronic devices. The board consists of an Atmega328 microcontroller that can be programmed to control various hardware components, making it ideal for a variety of applications in IoT, robotics, home automation, and more.

In this experiment, we will explore how to use an Arduino Uno to display real-time temperature readings from a TMP36 temperature sensor on a 16x2 LCD display. The main goal is to read the analog temperature data, convert it into a human-readable temperature format, and display it on the LCD screen.

Arduino Uno Overview

The **Arduino Uno** is one of the most popular boards in the Arduino family due to its simplicity, low cost, and ease of use. It features:

- **Microcontroller:** Atmega328P
- **Operating Voltage:** 5V
- **Digital I/O Pins:** 14 (of which 6 can be used as PWM outputs)
- **Analog Input Pins:** 6
- **Flash Memory:** 32 KB
- **Clock Speed:** 16 MHz
- **USB Connection:** For programming and communication
- **Power Supply:** Can be powered via USB or an external power supply

The Arduino Uno can be programmed using the Arduino IDE, which allows users to write and upload code that will control various electronic components connected to the board.

Components Used in This Experiment

1. Arduino Uno Board:

- Acts as the central controller that processes the sensor data and drives the LCD.

2. HC-SR04 Ultrasonic Sensor

- Used to detect the presence of a glass by measuring the distance from an object.
- Works by sending out an ultrasonic sound wave and calculating the time it takes for the echo to return.
- The sensor consists of a **trigger pin** (to send signals) and an **echo pin** (to receive reflected signals).

3. Servo Motor:

- A motor that moves to specific angles (e.g., 0°, 45°, 90°) based on signals from the Arduino.
- It is used to control the dispensing mechanism when a glass is detected.

4. Jumper Wires:

- Used to connect the components (Arduino, sensor, and servo motor) together on a breadboard or circuit.

5. Breadboard:

- A tool for creating prototype circuits without soldering. It provides a simple way to build the connections between components.
-

Code Explanation

The following code reads the ultrasonic sensor's distance measurement, checks if an object (glass) is within a certain range, and controls the servo motor accordingly.

```
#include <Servo.h>

// Pin definitions
const int trigPin = 10;    // Ultrasonic sensor trigger pin
const int echoPin = 9;     // Ultrasonic sensor echo pin
Servo myServo;             // Create servo object

// Constants for distance thresholds
const int glassDetectDistance = 20; // Detection threshold (20 cm)
const int hysteresis = 2;           // Buffer to avoid rapid switching

void setup() {
    pinMode(trigPin, OUTPUT);
```

```

    pinMode(echoPin, INPUT);
    myServo.attach(7); // Attach servo to pin 7
    myServo.write(90); // Start at 90° (default position)
    Serial.begin(9600); // Start serial communication
    delay(1000);        // Allow the system to stabilize
}

void loop() {
    int distance = getUltrasonicDistance();

    Serial.print("Distance: ");
    Serial.print(distance);
    Serial.println(" cm");

    // Add hysteresis to prevent rapid toggling
    if (distance > 0 && distance <= (glassDetectDistance - hysteresis))
    {
        myServo.write(45); // Move to 45° (dispense position)
        Serial.println("Glass detected. Servo rotated to 45 degrees.");
    } else if (distance > (glassDetectDistance + hysteresis)) {
        myServo.write(90); // Return to 90° (default position)
        Serial.println("No glass detected. Servo at 90 degrees.");
    }

    delay(200); // Slight delay for stability
}

// Function to measure distance using the ultrasonic sensor
int getUltrasonicDistance() {
    long duration;
    int distance;

    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    duration = pulseIn(echoPin, HIGH);
    distance = (duration / 2.0) * 0.034; // Convert time to cm

    return distance;
}

```

Code Breakdown

1. Libraries:

- The Servo.h library is included to allow easy control of the servo motor.

2. Pin Setup:

- The ultrasonic sensor is connected to pins 10 (trigger) and 9 (echo).
- The servo motor is connected to pin 7.

3. Initialization:

- The servo motor starts at 90° (idle position).
- Serial communication is started for debugging.

4. Sensor Reading:

- The function `getUltrasonicDistance()` calculates the distance from the object using the ultrasonic sensor.

5. Hysteresis for Stability:

- To prevent rapid toggling of the servo, a buffer zone (hysteresis) is added.
- The servo moves only if the distance changes beyond a threshold.

6. Servo Control Based on Detection:

- If an object (glass) is detected within 20 cm, the servo moves to 45° (dispense position).
- If no object is detected, the servo returns to 90°.

7. Delay for Stability:

- A small delay (`delay(200);`) is added to ensure smooth operation.

Conclusion

This experiment demonstrates how to use the Arduino Uno with an ultrasonic sensor and a servo motor to create an automated water dispensing system. By combining the Arduino's processing capabilities with simple hardware components like the HC-SR04 ultrasonic sensor and a servo motor, we can design an efficient touchless dispensing system.